

Worker Management Android Application

IIT Tirupati

Pranay, Poojitha, Mounika, Neha, Srivally, Yuvraj

September 02, 2024

Contents

1	Introduction	2
2	Project Description	2
3	Tech Stack	2
4	Architecture	3
5	Code Details	4
6	User Manual	5
7	Developer Manual	5
7.1	How to Run the Project	5
7.2	How to Create APK from WorkerManagement Folder	6
7.3	Adding Features and Best Practices	6
8	Conclusion	6

1 Introduction

The Worker Management application is a mobile Android app designed to connect unskilled workers with temporary job opportunities posted by managers or organizations. It addresses sustainable development goals such as reducing poverty and improving employment access by facilitating local, fair-wage job matching without intermediaries.

The app features two user roles: workers, who can register, log in, and browse available jobs; and managers, who can register, log in, post jobs, manage (view/delete) postings. Built using modern Android tools, it leverages Firebase for real-time data handling.

Key functionalities include user authentication (custom username/password), job posting with details like type, wage, location (pin), and real-time job listings via RecyclerViews. The project emphasizes simplicity, real-time synchronization, and user-friendly interfaces.

This report covers the tech stack, architecture, code details, user manual, and developer manual, including setup, running the project, and APK generation. The source code is structured in the WorkerManagement folder, with standard Android file organization.

2 Project Description

Developed a full-stack Android mobile application, "Worker Management," using Kotlin and Android Studio to connect unskilled workers with temporary job opportunities posted by managers/organizations. Implemented user authentication and registration for two roles (workers and managers) with Firebase Realtime Database for secure data storage and real-time synchronization. Designed interactive UI with XML layouts, Data Binding, and RecyclerView to display dynamic job listings, enabling workers to browse available works and managers to post, view, and delete job postings. Integrated Firebase queries (orderByChild, equalTo) for efficient data retrieval, duplicate checks, and real-time updates across app screens. Utilized Shared-Preferences for session management and ensured case-sensitive credential handling with proper validation and feedback via Toast messages. Focused on sustainable development goals by facilitating local, fair-wage employment, resulting in a user-friendly app optimized for real-time job matching.

3 Tech Stack

The Worker Management app is built using a modern Android development stack focused on simplicity, real-time data handling, and ease of maintenance. Here's a breakdown:

- **Programming Language:** Kotlin (primary language for all business logic, UI interactions, and Firebase integrations). Kotlin is used for its conciseness, null safety, and interoperability with Java.
- **IDE/Development Environment:** Android Studio (version Bumblebee or later, given the use of Kotlin and Compose elements).
- **UI Framework:**
 - XML Layouts (View-based system) for most activities (e.g., activity_login.xml, activity_manage.xml).
 - Jetpack Compose for theming and some UI elements (e.g., Color.kt, Theme.kt, Type.kt in ui/theme package). This hybrid approach allows for declarative UI in themes while sticking to imperative XML for core layouts.
- **Database and Backend:** Firebase Realtime Database (for storing user data, worker/manager profiles, and job postings). It's used for real-time syncing, queries (e.g., orderByChild, equalTo), and CRUD operations. No custom backend server; Firebase handles authentication implicitly through database rules (though explicit auth isn't implemented in code—logins are username/password based without Firebase Auth).
- **Libraries and Dependencies** (inferred from build.gradle.kts and code):

- Firebase Database SDK (com.google.firebase:firebase-database).
- RecyclerView (androidx.recyclerview:recyclerview) for displaying job lists.
- Data Binding (enabled in build.gradle.kts) for binding UI elements (e.g., ActivityLoginBinding).
- Kotlin Coroutines or standard callbacks for async operations (Firebase uses ValueEventListener).
- No explicit networking libraries since Firebase handles it.
- Other potential deps: androidx.appcompat, androidx.constraintlayout for layouts.
- **Version Control:** GitHub (for storing project files and documentation).
- **Testing:** Basic unit tests (ExampleUnitTest.kt) and instrumented tests (ExampleInstrumentedTest.kt) using JUnit and AndroidX Test.
- **Build System:** Gradle (Kotlin DSL via build.gradle.kts), with wrappers (gradlew, gradlew.bat) for building APKs.
- **Target Platforms:** Android (min SDK 21+ based on code; tested on Pixel 3 API 35 emulator). Supports various screen sizes, but UI may vary due to fixed designs.
- **Other Tools:**
 - Google Services (google-services.json for Firebase integration).
 - ProGuard (proguard-rules.pro for obfuscation in release builds).

This stack is lightweight, cost-effective (Firebase free tier), and suitable for a real-time app like this, focusing on quick development without heavy server-side logic.

4 Architecture

The app follows a MVVM-like (Model-View-ViewModel) architecture with some MVC elements, typical for small Android apps using Firebase. It's activity-based (no fragments except a mention in Manage.kt, but not used). Key components:

- **Layers:**
 - **Model:** Data classes like WorkerData.kt, ManagerData.kt, WorkData.kt represent entities stored in Firebase. Firebase DatabaseReference handles data persistence and queries.
 - **View:** XML layouts inflated via Data Binding (e.g., ActivityLoginBinding). RecyclerViews with adapters (AdapterClassManage.kt, AdapterClassSearch.kt) for dynamic lists.
 - **ViewModel/Controller:** Activities (e.g., Login.kt, Manage.kt) act as controllers, handling UI logic, Firebase listeners, and intents. No explicit ViewModels (from Jetpack), but data binding and event listeners simulate this.
 - **Repository/Data Layer:** Direct Firebase interactions in activities (e.g., databaseReference.orderByChild().equalTo() for queries). No separate repository class, keeping it simple but less modular.
- **Data Flow:**
 - Real-time syncing via Firebase ValueEventListener (e.g., in Manage.kt and Search.kt to fetch and update job lists).
 - CRUD Operations:
 - * Create: Push new keys in Firebase (e.g., Register1.kt: databaseReference1.push().key).
 - * Read: Queries with orderByChild/equalTo (e.g., login checks for existing users).
 - * Update: Not implemented (e.g., no edit functionality).
 - * Delete: In AdapterClassManage.kt via removeValue() on item click.

- Authentication: Custom username/password check against Firebase data (insecure; no hashing or Firebase Auth). Stores manager name in SharedPreferences for session-like behavior.
- **Navigation:** Intent-based between activities (e.g., from MainActivity to Login.kt). No Navigation Component; simple linear flows (e.g., Register → Login → Search/Manage).
- **UI Patterns:**
 - RecyclerView with LinearLayoutManager for lists.
 - Toasts for feedback, basic error handling via onCancelled.
 - SharedPreferences for lightweight session data (e.g., manager name).
- **Scalability & Limitations:**
 - Pros: Real-time updates, easy to extend with more Firebase features (e.g., Auth, Storage for images).
 - Cons: No offline support (Firebase offline not enabled), potential security issues (plain passwords), no pagination for large lists, emulator-specific UI.

Data Flow Diagram (Textual):

```
User -> MainActivity (Worker/Manager Portal)
      Worker Flow: Login/Register1 -> Search (Job List via RecyclerView)
      Manager Flow: Login2/Register2 -> Manage (Posted Jobs via RecyclerView) -> PostWork (Add Job)
All Flows -> Firebase Realtime DB (workers/, managers/, work/)
```

5 Code Details

The code is organized in a standard Android structure (app/src/main/java/com/example/workermanagement). Key highlights:

- **Key Classes:**
 - **MainActivity.kt:** Entry point with buttons to choose worker/manager portal.
 - **Login.kt & Login2.kt:** Handle worker/manager login with Firebase queries. Use Data Binding for UI.
 - **Register1.kt & Register2.kt:** Worker/manager registration; check for duplicates before pushing to DB.
 - **Search.kt:** Displays all jobs in a RecyclerView using AdapterClassSearch.kt.
 - **Manage.kt:** Manager dashboard; shows posted jobs, allows deletion via AdapterClassManage.kt. Uses SharedPreferences for name.
 - **PostWork.kt:** Form to post new jobs; checks for duplicates by type.
 - **Adapters:** AdapterClassManage.kt (with delete), AdapterClassSearch.kt (read-only). Use ViewHolder pattern.
 - **Data Models:** WorkerData.kt, ManagerData.kt, WorkData.kt (simple data classes).
 - **UI Theme:** Compose-based in ui/theme (Color.kt, Theme.kt) for custom colors/fonts.
- **Firebase Integration:**
 - Instance: FirebaseDatabase.getInstance().
 - Paths: "workers/", "managers/", "work/".
 - Queries: orderByChild("field").equalTo(value) for uniqueness/login.
 - Listeners: ValueEventListener for real-time updates (addValueEventListener in lists).

- **Error Handling:** Basic Toasts for errors; logs in onCancelled.
- **Security Notes:** Passwords stored in plain text—recommend hashing (e.g., via SHA-256) or switching to Firebase Auth.
- **Code Quality:**
 - Pros: Clean, modular adapters; uses Kotlin features like null safety.
 - Improvements: Add coroutines for async, implement proper auth, use ViewModels for better separation.

Build & Config Files: build.gradle.kts enables data binding; google-services.json for Firebase.

6 User Manual

The opening screen contains two options which lead to portal for workers and portal for managers.

On clicking the worker portal a login screen for workers is available. If already there is an account people can login using those credentials and if not they can register a new account using the register button. On logging in, a list of jobs or works are available for the worker out of which the worker can access the details of his interested work.

On clicking the manager portal a login screen for managers is available. If already there is an account people can login using those credentials and if not they can register a new account using the register button. On logging in, the manager gets a screen where he can see all the jobs or works posted by him. He can delete the posts by delete button. He can also add posts by using the add button and filling out the required details.

Note - In login screen while entering the credentials, small letters and capital letters appear as capital letters. But, the app is case sensitive and small letters, capital letters are stored case sensitively in database.

A common database is used to store the data which can be accessed by the portals with respect to the need of the worker or the organization.

There is an option for the organization in its part of the app to post new work with details such as type of workers needed, wages for the work and location of the work.

The worker has a screen where the worker can get the list of works by their choice which may be by type of work or by wages or by location.

The front end of the project has an interactive User Interface (UI) so that the android app can be accessed and used comfortably. It has login authentication for both workers and organizations. There is an option to create an account if account is not registered.

The user interface is designed primarily for Pixel 3 API 35 emulator. So inconsistency in user interface can be seen in devices of other sizes.

Note for managers – The organization name with which the manager is registering must be used as the organization name at the time of posting new works.

7 Developer Manual

7.1 How to Run the Project

To run the project for development:

1. Clone the repository from GitHub: `git clone [repo-url]`.
2. Open the WorkerManagement folder in Android Studio.
3. Set up Firebase: Create a Firebase project, enable Realtime Database, download google-services.json, and place it in app/.
4. Sync Gradle: Click "Sync Now" in Android Studio or run `./gradlew build`.

5. Run the app: Select an emulator (e.g., Pixel 3 API 35) or connected device and click Run.
6. For testing: Run unit tests via `./gradlew test`; instrumented tests via `./gradlew connectedAndroidTest`.

Prerequisites: Android Studio installed, Android SDK with API 21+, Java/Kotlin setup.

7.2 How to Create APK from WorkerManagement Folder

To generate an APK:

1. Open the project in Android Studio.
2. Build the project: Go to Build $\hookrightarrow$ Build Bundle(s)/APK(s) $\hookrightarrow$ Build APK(s).
3. For signed APK (release): Build $\hookrightarrow$ Generate Signed Bundle/APK $\hookrightarrow$ APK $\hookrightarrow$ Create or select keystore $\hookrightarrow$ Fill details $\hookrightarrow$ Select debug/release variant $\hookrightarrow$ Finish.
4. Locate APK: In `app/build/outputs/apk/debug/` or `release/` (e.g., `app-debug.apk`).
5. Via command line: Navigate to WorkerManagement, run `./gradlew assembleDebug` for debug APK or `./gradlew assembleRelease` for release (requires signing config in `build.gradle.kts`).

The APK can be installed on Android devices via ADB or file transfer.

7.3 Adding Features and Best Practices

- New Activity: Create XML layout, Kotlin class, add to Manifest. - Firebase: Use DatabaseReference for paths; add listeners carefully to avoid leaks (remove onDestroy). - UI: Use Data Binding for inflation; RecyclerView for lists. - Best Practices: Secure passwords with hashing or Firebase Auth; enable offline persistence (`database.keepSynced(true)`); add pagination for large data. - Known Issues: Insecure auth; no input validation; UI inconsistencies on non-Pixel devices. - Deployment: Generate signed APK; upload to Google Play Store.

8 Conclusion

The Worker Management app demonstrates effective use of Android and Firebase for real-time job matching. Future enhancements could include proper authentication, multi-language support, and advanced features like notifications. This project showcases skills in mobile development, database integration, and user-centric design.